

AUTO TICKET CLASSIFICATION USING FLOW DESIGNER

ServiceNow Incident Management Project

PACADEMIC YEAR	2024–2028
Prepared By	Vaishnavi Chinna
Role	Team Leader
Department	B.Tech Information Technology
Institution	Sri Ramanujar Engineering College

A structured technical documentation of project ideation, requirements, architecture, implementation approach, testing, results, limitations and future enhancement opportunities.

EXECUTIVE. SUMMARY

Auto Ticket Classification using Flow Designer is documented as a ServiceNow Incident Management project focused on improving the handling of incoming support incidents through a structured, automated workflow. The project addresses the operational challenge of processing incident information consistently and reducing repetitive manual activities.

The documentation follows a complete project lifecycle: problem identification, ideation, requirement analysis, solution design, planning, testing, results, limitations and future scope. The report is intended to provide reviewers with a clear understanding of what the system is expected to accomplish and how its components fit together.

The project is positioned within the ServiceNow platform and uses Flow Designer as the workflow automation layer. The final implementation should be evaluated against the defined functional and non-functional requirements, with actual measured results recorded in the testing and results sections.

- Automate repeatable incident-management activities.
- Support consistent processing of ticket information.
- Improve traceability through defined requirements and test cases.
- Provide a scalable foundation for additional classification and routing logic.
- Maintain clear documentation for development, testing and future maintenance.

1. INTRODUCTION

Incident-management environments receive a continuous stream of service requests and incidents. When ticket handling relies heavily on manual review, the process can become time-consuming and inconsistent. A structured automation workflow can standardize repetitive steps and make the movement of information easier to track.

This project explores an automated ticket-classification workflow using ServiceNow Flow Designer. The documentation treats the workflow as the central orchestration layer connecting ticket information, decision logic, assignment or categorization actions, and resulting incident updates.

The report is designed to document not only the final workflow but also the reasoning behind the requirements, architecture, testing strategy and future improvements.

1.1. Project Objectives

- Define a clear and traceable workflow for ticket classification.
- Reduce repetitive manual handling during incident processing.
- Apply consistent business rules to incoming ticket information.
- Improve visibility of processing status and resulting actions.
- Validate the workflow through functional and performance-oriented testing.
- Provide an extensible design for future AI/ML-based classification.

1.2. Scope

The project scope covers the design and documentation of an automated ticket-classification workflow within ServiceNow. It includes requirement definition, workflow design, data movement, testing and result documentation.

Actual integrations, production deployment, model selection and measured performance values should be documented only after they are implemented and verified.

2. IDEATION PHASE

The ideation phase converts the operational problem into a clearly defined project opportunity. The emphasis is on understanding the user or support team's difficulty before selecting the technical solution.

2.1. Problem Statement

Support teams may need to manually inspect ticket details, identify the relevant category or routing information, and perform repetitive updates. This can increase handling time and introduce inconsistencies when similar tickets are processed differently.

There is a need for a structured mechanism that can interpret available ticket information and trigger consistent classification or routing actions through an automated workflow.

2.2. Empathy Map

Says: Users may request faster ticket handling, correct categorisation and clear status visibility.

Thinks: Users expect repetitive ticket operations to be automated and predictable.

Does: Users submit tickets, review information, update fields and follow the progress of incidents.

Feels: Manual repetition can create frustration, especially when ticket volumes increase.

Pains: Repetitive work, inconsistent categorisation, delays and avoidable manual errors.

Gains: Faster processing, consistent classification, traceability and reduced operational effort.

2.3. Brainstorming & Idea Selection

The team can use group discussion, mind mapping and impact-versus-effort prioritisation to identify possible approaches. Candidate approaches include rule-based workflow automation, scripted classification, and future ML-assisted classification.

For the current project documentation, Flow Designer is the workflow-orchestration approach because it provides a structured way to connect triggers, conditions and actions.

3. REQUIREMENT ANALYSIS

Requirements define what the system must accomplish and the quality levels expected from it. Each requirement should have a unique identifier so that it can be traced from design through testing.

3.1 Functional Requirements

ID	Requirement	Description
FR-01	Ticket Intake	Receive or detect an incident/ticket that requires processing.
FR-02	Data Validation	Validate the availability of required ticket information before processing.
FR-03	Classification Logic	Evaluate ticket information using the configured classification rules or logic.
FR-04	Field Update	Update relevant incident fields based on the classification outcome.
FR-05	Routing / Assignment	Route or assign the ticket according to configured workflow conditions.
FR-06	Status Handling	Maintain a clear processing status and outcome.
FR-07	Error Handling	Handle missing, invalid or unexpected inputs without silently failing.
FR-08	Auditability	Maintain sufficient traceability for workflow execution and resulting updates.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Workflow configuration and outputs should be understandable to authorised users.
NFR-02	Reliability	Workflow failures should be handled gracefully and remain traceable.
NFR-03	Performance	Processing should complete within an acceptable operational response time.
NFR-04	Security	Access and data handling should follow the configured ServiceNow security model.
NFR-05	Scalability	The design should support increased ticket volume and additional rules.
NFR-06	Maintainability	Rules and workflow steps should be modular and easy to update.

4. SYSTEM DESIGN

The system design describes how ticket information moves through the automated process. The architecture should separate input handling, decision logic, record updates and external or supporting services so that individual components can be maintained independently.

4.1. Proposed Workflow Architecture

- **Trigger Layer:** starts the flow when the defined ticket/incident condition occurs.
- **Input Validation:** checks that required fields are available and usable.
- **Classification Layer:** evaluates ticket attributes against configured rules or classification logic.
- **Decision Layer:** determines the next action based on the classification result.
- **Action Layer:** updates fields, assignment information or other supported incident attributes.
- **Audit / Notification Layer:** records outcomes and optionally informs relevant users.

4.2. Data Flow

At a high level, the flow receives incident information, validates it, processes classification logic, generates a classification outcome, and applies the appropriate workflow action. The resulting incident record becomes the main output of the process.

A detailed Level-1 Data Flow Diagram should identify external actors, the ServiceNow incident record, Flow Designer, classification logic, data stores and final outputs.

4.3. Technology Stack

Platform: ServiceNow Incident Management

Automation: ServiceNow Flow Designer

Workflow Elements: Triggers, conditions, actions and decision branches as configured in the project

Data: ServiceNow incident/ticket records and associated fields

Documentation & Testing: Test cases, execution evidence and result records maintained as part of the project report.

4.4. Design Principles

- **Traceability:** every major requirement should map to one or more test cases.
- **Modularity:** classification and action logic should be separated where practical.
- **Fail-safe behaviour:** invalid or incomplete input should produce a controlled outcome.
- **Maintainability:** workflow steps should be named clearly and documented.
- **Extensibility:** the design should allow future classification rules or ML integration.

5. PROJECT PLANNING & SCHEDULING

The project should be executed in controlled phases so that requirements, design, workflow configuration, testing and documentation are completed systematically.

Phase	Primary Activities	Owner	Deliverable
1. Requirement Analysis	Problem definition, scope, FR/NFR	Team Lead / Team	Requirement baseline
2. Design	Workflow, data flow, architecture	Team Lead / Team	Design specification
3. Configuration	Flow Designer workflow and rules	Assigned Members	Working workflow
4. Testing	Functional, negative and performance checks	Testing Team	Test report
5. Validation	Review outputs and fix defects	Team Lead / Team	Validated build
6. Documentation	Screenshots, results, appendix	Team Lead / Team	Final project report

5.1. Team Leadership & Responsibility

As **Team Leader – Vaishnavi C**, the role includes coordinating the project workflow, maintaining requirement and documentation consistency, tracking task completion, supporting issue resolution, and ensuring that the final report reflects verified project outputs.

Individual team-member responsibilities should be recorded separately so that ownership of configuration, testing, documentation and presentation activities remains traceable.

5.2. Risk Management

- Incomplete ticket data → introduce validation and controlled fallback handling.
- Incorrect classification rule → validate against representative test scenarios.
- Workflow execution failure → maintain execution logs and error-handling paths.
- Changing requirements → maintain versioned requirement and workflow documentation.
- Limited test data → document dataset limitations and avoid unsupported performance claims.

6. TESTING & VALIDATION

Testing verifies whether the workflow behaves according to its requirements. The test strategy should include positive, negative, boundary and failure-oriented scenarios. Evidence such as screenshots, execution logs and test results should be retained for review.

6.1 Functional Test Cases

ID	Scenario	Input	Expected Result	Actual Result	Status
TC-01	Valid ticket input	Valid incident details	Correct classification and workflow action	Record actual result	Pass/Fail
TC-02	Missing required field	Incomplete incident data	Controlled validation/fallback	Record actual result	Pass/Fail
TC-03	Alternative category	Ticket matching another rule	Correct alternative classification	Record actual result	Pass/Fail
TC-04	Unrecognised input	Ticket not matching rules	Defined fallback behaviour	Record actual result	Pass/Fail
TC-05	Repeated execution	Multiple similar tickets	Consistent processing	Record actual result	Pass/Fail
TC-06	Workflow failure	Forced/observed failure condition	Error is traceable and controlled	Record actual result	Pass/Fail
TC-07	High-volume scenario	Representative batch	Acceptable processing time	Record measured value	Pass/Fail

6.2. Performance Validation

- Measure average workflow execution time across representative tickets.
- Record error/failure rate and identify the reason for each failure.
- Measure behaviour under increasing ticket volume where feasible.
- Record actual values; do not replace measured results with target values.

6.3. Defect Management

Each defect should be logged with an identifier, description, severity, reproduction steps, expected behaviour, actual behaviour, owner, resolution and retest status.

7. RESULTS & OUTPUT DOCUMENTATION

The results section should provide direct evidence that the implemented workflow behaves as intended. Screenshots should be presented in the same sequence that a user or reviewer would experience the workflow.

Evidence	What to Capture
Flow Designer canvas	Complete workflow with trigger, conditions, actions and decision branches visible.
Input ticket	Representative incident/ticket before processing.
Classification result	Classification/category or relevant output produced by the workflow.
Updated ticket	Incident after automated field updates or assignment.
Execution evidence	Flow execution details, logs or run history where available.
Exception scenario	Controlled handling of invalid or unsupported input.

7.1. Result Summary

The final report should state the number of requirements verified, the number of test cases passed, the measured workflow response time, and any known limitations. These values must be filled from actual project execution evidence.

8. ADVANTAGES & LIMITATIONS

The project should be evaluated objectively. Automation can reduce repetitive effort and improve consistency, but the quality of outcomes depends on the completeness of ticket information, correctness of configured logic and the maintainability of the workflow.

Advantages	Limitations / Considerations
Reduces repetitive manual processing.	Requires accurate configuration and maintenance.
Provides consistent workflow execution.	Unexpected ticket patterns may require new rules.
Improves traceability of automated actions.	Workflow complexity can grow as rules increase.
Supports structured incident handling.	Performance must be validated with realistic volumes.
Can be extended with advanced classification.	AI/ML integration requires suitable data and validation.

9. CONCLUSION

The project presents a structured approach to automating ticket-classification activities within ServiceNow using Flow Designer. The documentation connects the operational problem with requirements, workflow design, testing and evidence-based results.

The project also establishes a foundation for future improvements. The quality of the final outcome should be judged using actual test evidence, measured performance and traceability between requirements and implemented behaviour.

10. FUTURE SCOPE

- Introduce ML-assisted classification when a suitable labelled dataset is available.
- Expand classification categories and routing rules based on historical ticket patterns.
- Add analytics dashboards for classification accuracy, processing time and exception trends.
- Integrate additional notification and escalation mechanisms.
- Improve security controls and audit visibility where required.
- Evaluate cloud-scale deployment and higher-volume processing.
- Continuously refine rules using validated operational feedback.

11. APPENDIX

The appendix should contain the supporting artefacts required to reproduce, review or validate the project.

Appendix	Artefact	Details
A	Source / Configuration	Key Flow Designer configuration, scripts or configuration references.
B	Test Evidence	Screenshots, execution results and defect records.
C	Dataset / Sample Tickets	Dataset name, source, size and description, if applicable.
D	Repository	GitHub repository or institutional repository link, if applicable.
E	Demo	Project demonstration video or live deployment link, if available.
F	References	Official ServiceNow documentation and other sources used.

11.1. References

ServiceNow platform and Flow Designer official documentation should be cited using the exact pages consulted during implementation.

Any dataset, research paper, third-party integration or technical reference used by the team should be listed with author/organisation, title, year and access information.

PROJECT LEAD

Vaishnavi Chinna

Team Leader – B.Tech Information Technology